DOCUMENTACION DE SISTEMA

RootCause Bitcoin Defense

12 · Pruebas y calidad

Version analizada 0.3.0 · commit acdf4dd · generado el 2026-08-27
Documento 12 de la serie · fuente: docs/system-documentation/12-testing-and-quality.md

Contenido

1 · Herramientas
2 · Inventario de pruebas
3 · Cobertura por módulo
4 · Lo que las pruebas cubren especialmente bien
5 · Las puertas de calidad
6 · Criterios de aceptación observados
7 · Casos límite cubiertos por prueba
8 · Huecos de prueba y propuesta priorizada
9 · Fixtures y datos de prueba
10 · Reproducir la validación completa en local

Qué se prueba, con qué herramienta, qué queda sin probar y cuáles serían las pruebas siguientes por orden de valor.

> Complementa a `../TESTING.md`, que explica la estrategia. Aquí está el **inventario medido**: cobertura por módulo, huecos concretos y una propuesta priorizada.

---

1 · Herramientas

Capa	Herramienta	Nota
Pruebas unitarias y de integración	`node:test` y `node:assert/strict`	**Del runtime.** Sin Jest, Mocha ni Vitest
Cobertura	No configurada	`.gitignore` reserva `coverage/`, pero nada la genera
Análisis estático	CodeQL con `security-extended`	En GitHub, semanalmente
Lint de JavaScript	Ninguno	No hay ESLint ni Prettier
Comprobación de sintaxis	`node --check` sobre cada `.js`	Dentro de `validate-repo.js`
Formato	`.editorconfig`	Sin verificador automático de código
Lint de YAML y workflows	`zizmor`	En CI
Lint de Dockerfile	`hadolint`	En CI

Capa	Herramienta	Nota
Lint de documentación	`check-doc-style.js`	**Propio del proyecto**
Secretos	`check-history-secrets.js` + `gitleaks`	Histórico completo

> La ausencia de ESLint es coherente con el proyecto —sería una dependencia— y deja un hueco real: nadie detecta una variable sin usar o una promesa sin `await`. Está en la propuesta de la sección 8.

---

2 · Inventario de pruebas

64 casos en 11 archivos, todos pasando.

Archivo	Casos	Tipo
`test/incident-lifecycle.test.js`	14	Unitaria de servicio + arranque
`test/attack-patterns.test.js`	11	Unitaria de dominio
`test/posture.test.js`	9	Unitaria de dominio
`test/rules-catalog.test.js`	6	**De consistencia** entre código y documentación
`test/api.test.js`	4	**Integración con servidor HTTP real**
`test/rule-engine.test.js`	4	Unitaria de dominio
`test/bitcoin-rpc.test.js`	3	Unitaria con `fetch` inyectado
`test/encrypted-store.test.js`	3	Unitaria con sistema de archivos real
`test/secret-guard.test.js`	3	Unitaria de dominio
`test/firmware.test.js`	3	Unitaria de dominio
`test/desktop-launch.test.js`	2	Unitaria de arranque
`test/audit-log.test.js`	2	Unitaria de infraestructura

Ejecución completa en unos 5 segundos. **No hay pruebas lentas, ni flaky, ni dependientes de red.**

---

3 · Cobertura por módulo

Medida por lectura, no por instrumentación: no hay herramienta de cobertura configurada.

Módulo	Cobertura aparente	Casos que lo tocan
`src/domain/rule-engine.js`	[OK] **Alta**	15 casos entre `attack-patterns` y `rule-engine`
`src/domain/posture.js`	[OK] **Alta** — los 8 controles	9 casos
`src/domain/firmware.js`	[OK] Alta	3 casos + indirecta desde el motor

Módulo	Cobertura aparente	Casos que lo tocan
`src/domain/secret-guard.js`	[OK] Alta	3 casos + 5 cargas hostiles en `check-security-claims.js`
`src/infrastructure/encrypted-store.js`	[OK] Alta	3 casos, incluida la clave incorrecta
`src/infrastructure/audit-log.js`	[OK] Media-alta	2 casos: cadena íntegra y manipulada
`src/infrastructure/bitcoin-rpc.js`	[ATENCION] Media	3 casos; **el camino de error de `snapshot()` no se prueba**
`src/app.js`	[ATENCION] Media	Cubierto por `api.test.js` en sus rutas felices y en el 403
`src/api/router.js`	[ATENCION] Media	Indirecta; **9 de 15 endpoints sin prueba**
`src/services/defense-service.js`	[OK] **Media-alta**	12 pruebas directas sobre `mergeIncidents` y 2 sobre `initialize`, más la cobertura indirecta por la API
`src/domain/risk.js`	[FALLO] **Sin prueba directa**	Solo indirecta
`src/config.js`	[FALLO] **Sin prueba directa**	Se usa en las pruebas, no se prueba
`src/services/watchtower.js`	[FALLO] **Sin prueba**	—
`src/services/demo-state.js`	[ATENCION] Indirecta	Es el fixture de casi todo
`src/web/static/app.js`	[FALLO] **Sin prueba**	601 líneas de JavaScript de navegador
`scripts/*`	[FALLO] **Sin prueba**	2698 líneas; se validan ejecutándolos en CI

---

4 · Lo que las pruebas cubren especialmente bien

4.1 · Simetría en los patrones de ataque

Cada detección tiene su **par negativo**, que es lo que impide que la regla degenere en un detector que siempre dispara:

Detecta	Y comprueba que no dispara con
Barrido del 100 % del saldo	Un pago parcial del 20 %
Ráfaga de gastos en menos de una hora	Dos gastos separados por horas
Filtración de un fabricante que usas	Filtración de un fabricante que no usas
Matriz de avisos vieja	Matriz verificada recientemente
Quorum 3-de-3	Quorum 2-de-3
Bind fuera de loopback	Valores por defecto
Clave de datos débil	Clave generada

4.2 · Consistencia entre código y documentación

`test/rules-catalog.test.js` extrae los códigos `BTC-*` del motor con una expresión regular sobre el propio fuente y los compara con el catálogo, en las dos direcciones:

Prueba	Qué impide
Toda regla del motor tiene ficha	Añadir una regla sin documentarla
Toda ficha corresponde a una regla real	Documentar una regla que no existe
Cada ficha declara todo lo necesario	Una ficha incompleta que no se puede defender
Las severidades usan el vocabulario del proyecto	Inventar una severidad
Cada familia tiene al menos una regla	Una familia vacía en el índice
Las reglas relacionadas existen	Un enlace roto entre fichas

> **Es la prueba más inusual del repositorio.** Convierte «la documentación está al día» en una condición del build, no en una buena intención.

4.3 · Integración con un servidor real

`test/api.test.js` levanta un `http.Server` en el puerto 0 con un `MemoryStore` y un `DefenseService` reales. **No hay mocks de la capa HTTP.** Cubre:

Caso	Comprueba
`GET /api/health` y `/api/summary`	Contrato de respuesta
`POST /api/wallets` sin cabecera	`403`
`POST /api/wallets` con campo secreto	`422`
`POST /api/wallets` con procedencia pública	`201` y reanálisis

---

5 · Las puertas de calidad

Cinco en CI, más una de auditoría con seis trabajos.

Puerta	Trabajo de CI	Qué garantiza
Calidad	`calidad`	Validación y pruebas en **3 sistemas operativos × 2 versiones de Node**
Solo local	`solo-local`	Cero dependencias, cero orígenes remotos, CSP intacta, `npm ls` vacío
Invariantes	`invariantes`	**Arranca la aplicación real** y la ataca; más un `grep` contra métodos RPC de gasto
App de Windows	`app-windows`	Empaqueta, **arranca el `.cmd` real** y comprueba `/api/health`, el panel y **que el inventario no esté vacío**
Documentación	`documentacion`	Enlaces, estilo y fichas sincronizadas

Trabajo de auditoría	Qué garantiza
`custodia`	Las 14 etapas, con informe publicado como artefacto 90 días
`runtime`	Avisos del Node que **se distribuiría hoy**, no el del runner
`secretos`	Histórico completo, con patrones del dominio, más `gitleaks`
`workflows`	`zizmor` sobre los workflows y `hadolint` sobre el Dockerfile
`inteligencia`	Frescura de la matriz de avisos, **sin tumbar el build**
`plugin`	`plugin/` coincide exactamente con `.claude/`

5.1 · La matriz de la puerta de calidad

Sistema operativo	Node 22	Node 24
`windows-latest`	[OK]	[OK]
`ubuntu-latest`	[OK]	[OK]
`macos-latest`	[OK]	[OK]

Con `fail-fast: false`, para ver todos los fallos y no solo el primero. Windows va primero en la lista porque es el destino principal del producto.

---

6 · Criterios de aceptación observados

Deducidos del comportamiento de CI y del release, no de un documento:

Criterio	Dónde se exige
Las 64 pruebas pasan en 6 combinaciones	`ci.yml`
Cero dependencias declaradas o instaladas	`check-local-only.js`
CSP con sus 5 directivas clave, sin `unsafe-inline`	Ídem
Ningún método RPC de gasto en `src/`	`grep` en `ci.yml`
20 invariantes contra la aplicación en marcha	`check-security-claims.js`
Todo enlace de la documentación resuelve	`check-docs.js`
Estilo visual: un H1, tablas con cabecera, todo indexado	`check-doc-style.js`
Las 26 fichas coinciden con el catálogo	`generate-rule-docs.js --check`
El tag de release coincide con `package.json`	`release-windows.yml`
El artefacto instalado arranca con inventario	Ídem
El runtime empaquetado coincide con su SHA-256 oficial	`build-portable.ps1`
El paquete no incluye `data`, `.env`, `.git` ni `node_modules`	Ídem

> El criterio *«el artefacto instalado arranca con inventario»* merece atención: comprueba `summary.totals.wallets >= 1` sobre la aplicación **instalada de verdad** por el instalador silencioso. Es la forma de detectar el fallo clásico de un binario que compila, pesa lo esperado y se instala vacío.

---

7 · Casos límite cubiertos por prueba

Caso	Archivo
Clave de descifrado incorrecta	`encrypted-store.test.js`
El archivo cifrado no filtra metadatos en claro	Ídem
Cadena de auditoría con metadata alterada	`audit-log.test.js`
Campo secreto a profundidad arbitraria	`secret-guard.test.js`
`seedOrigin` y `firmwareAtSeedCreation` **no** se bloquean	Ídem
RPC remoto sin permiso explícito	`bitcoin-rpc.test.js`
Método fuera de la allowlist	Ídem
Sufijos de fabricante en versiones de firmware	`firmware.test.js`
Clave larga pero repetitiva	`posture.test.js`
Bind abierto **dentro** de un contenedor	Ídem
Sin configuración, la postura no inventa hallazgos	Ídem
El navegador no se abre sin petición explícita	`desktop-launch.test.js`
El navegador nunca se abre con bind no loopback	Ídem

---

8 · Huecos de prueba y propuesta priorizada

Ordenada por **riesgo si falla × probabilidad de que se rompa sin darse cuenta**.

#	Prueba propuesta	Módulo	Prioridad	Por qué
1	Concurrencia de `mutate()`: dos escrituras simultáneas no se pisan	`defense-service.js`	[FALLO] **Alta**	La cola es la única defensa contra la pérdida de escrituras, y nada la verifica
2	`riskScore` y `riskLevel` en sus fronteras (0, 35, 65, 85, 100)	`risk.js`	[ALTA] Media-alta	Es lo que ve el usuario y no tiene prueba directa
3	`loadConfig`: recorte de rangos, booleanos no-`true`, detección de contenedor	`config.js`	[ALTA] Media-alta	Gobierna todo, y sus caídas silenciosas son fáciles de romper
4	Los 9 endpoints sin prueba, incluidos `/api/posture`, `/api/audit` y `PATCH /api/incidents`	`router.js`	[ALTA] Media-alta	Más de la mitad del contrato HTTP
5	`Watchtower`: la guarda `running` y la absorción de errores	`watchtower.js`	[ALTA] Media	Sin ninguna prueba

#	Prueba propuesta	Módulo	Prioridad	Por qué
6	Límite de tasa: la petición 121 devuelve `429`	`app.js`	[ALTA] Media	Control declarado y no verificado por prueba
7	Límite de cuerpo: un cuerpo de 65 KiB + 1 devuelve `413`	`app.js`	[ALTA] Media	Ídem
8	`snapshot()` cuando el nodo falla a mitad de las cuatro llamadas	`bitcoin-rpc.js`	[ATENCION] Media-baja	El camino de error no se prueba
9	`EncryptedFileStore.load()` con archivo corrupto	`encrypted-store.js`	[ATENCION] Media-baja	Debe lanzar, no devolver estado vacío
10	`escapeHtml` y `safeUrl` del panel	`web/static/app.js`	[ATENCION] Media-baja	Requiere un entorno DOM; hoy nada cubre 601 líneas
11	Las 14 etapas de `audit-bitcoin.js` con estados forzados	`scripts/`	[ATENCION] Baja	Se valida ejecutándolo en CI

8.1 · Otras mejoras de calidad propuestas

#	Mejora	Prioridad
1	Activar `node --test --experimental-test-coverage` y publicar el porcentaje	[ALTA] Media
2	Un lint mínimo propio, sin dependencias, en la línea de los otros validadores	[ATENCION] Baja
3	Fijar la imagen base Docker por digest	[ATENCION] Baja
4	Registrar el `x-request-id` en el log de los 5xx para poder correlacionar	[ATENCION] Baja

---

9 · Fixtures y datos de prueba

Fixture	Dónde	Nota
`createDemoState()`	`src/services/demo-state.js`	Base de casi todas las pruebas
Aviso de prueba `BTC-ADV-TEST`	Dentro de `api.test.js`	Se define local, no se lee del archivo real
`examples/wallets.sample.json`	`examples/`	Huellas ficticias `A1000001`, `B2000002`, `C3000003`
`examples/event-unauthorized-spend.json`	Ídem	`txid` de 64 a
`examples/event-bulk-customer-access.json`	Ídem	Proveedor ficticio
Vector `xprv` de `check-security-claims.js`	Dentro del script	**Concatenado en tiempo de ejecución** para que el archivo no contenga una clave extendida literal que el validador tendría que denunciar

> El último detalle es la clase de cuidado que caracteriza al repositorio: el vector de prueba existe en ejecución pero **no existe en el archivo**, para no disparar su propio detector de secretos.

Todos los datos de prueba son **públicos y ficticios**. No hay ninguna huella, dirección ni saldo real en el repositorio.

---

10 · Reproducir la validación completa en local

```
node --test
node scripts/validate-repo.js
node scripts/check-local-only.js
node scripts/check-docs.js
node scripts/check-doc-style.js
node scripts/generate-rule-docs.js --check
node scripts/check-security-claims.js
node scripts/sync-plugin.js --check
```

Las que necesitan `git` o red:

```
node scripts/check-history-secrets.js
node scripts/check-runtime-advisories.js
node scripts/audit-bitcoin.js --report
```

Siguiente	
Despliegue y operación	`13-deployment-and-operations.md`
Volver al índice	`README.md`